



پروژه عملی درس برنامه‌سازی پیشرفته (C++)

دانشکده ریاضی و علوم کامپیوتر - دانشگاه علم و صنعت ایران

پروژه: آبشار جواهرات پیشرفته (Gem Cascade)

1. مقدمه و اهداف پروژه

هدف این پروژه، پیاده‌سازی نسخه تعاملی بازی پازل تطابقی (Match-3) در محیط کنسول به زبان **cpp** است. در این نسخه، تمرکز بر مفاهیم شیء‌گرایی، الگوریتم‌های بهینه‌سازی و جستجو (برای تشخیص بن‌بست و...)، مدیریت فایل و پردازش بازگشتی واکنش‌های زنجیره‌ای است.



2. قوانین و مکانیزم‌های اصلی بازی

2.1. مشخصات جدول بازی

- **ابعاد:** جدول بازی دارای ابعاد ثابت 8×8 است.
- **تنوع مهره‌ها:** حداقل ۵ نوع مهره متمایز با کاراکترها (و همچنین رنگ‌های مختلف) باید وجود داشته باشد.

- **تولید اولیه:** جدول در آغاز به صورت تصادفی پر می‌شود، اما نباید هیچ تطابق ۵ تایی یا بیشتر اولیه‌ای وجود داشته باشد. (یک تطابق n تایی مجموعه n تایی از یک کاراکتر یکسان است که در جدول با هم مجاور اند)

2.2. محدودیت حرکات و مدیریت بن‌بست (Deadlock)

- **حرکت مجاز:** تنها جابجایی دو مهره مجاور (افقی یا عمودی) مجاز است، به شرطی که منجر به ایجاد حداقل یک تطابق ۳ تایی شود. به عبارتی، جابجایی که مستقیماً منجر به هیچ امتیازی نشود غیرمجاز است.
- **حرکت نامعتبر:** اگر جابجایی هیچ تطابق ایجاد نکند، باید لغو شود و به کاربر اطلاع داده شود.
- **تشخیص بن‌بست:** پس از هر ریزش، تمام حرکات ممکن بررسی می‌شوند تا مشخص شود آیا حرکت مجازی باقی مانده است یا خیر.
- **بازتوزیع (Shuffle):** در صورت وقوع بن‌بست، جدول بدون کسر امتیاز به صورت تصادفی بازتوزیع می‌شود تا حداقل یک حرکت مجاز ایجاد شود.

2.3. جاذبه و واکنش‌های زنجیره‌ای (Recursion)

- پس از حذف مهره‌های منطبق، مهره‌های بالایی سقوط کرده و فضاهای خالی پر می‌شوند.
- خانه‌های خالی بالای صفحه با مهره‌های تصادفی جدید پر می‌شوند.
- در صورت ایجاد تطابق جدید پس از ریزش، فرآیند بررسی و حذف باید به صورت بازگشتی تکرار شود تا جدول پایدار شود.

3. قوانین خاص تطابق

- **تطابق 4 تایی (Match-4):** علاوه بر حذف 4 مهره، 4 خانه دیگر باقی مانده در همان سطر یا ستون حذف می‌شوند.
- **تطابق 5 تایی (Match-5):** تمام مهره‌های هم‌نوع در کل جدول حذف می‌شوند.

4. فروشگاه تجهیزات جانبی (Power-ups)

کاربر می‌تواند با خرج امتیازهای خود از قابلیت‌های زیر استفاده کند:

۱. **بمب (Bomb):** حذف محدوده 3×3 با مرکزیت خانه انتخابی. (120 امتیاز)
۲. **فش‌فشه / موشک (Rocket):** حذف کامل یک سطر یا ستون انتخابی. (100 امتیاز)
۳. **سیستم راهنما (Hint):** پیشنهاد بهترین حرکت ممکن بر اساس بیشترین امتیاز آتی با در نظر گرفتن تمام جواهرات موجود در جدول. (70 امتیاز)

هر قابلیت باید هزینه مشخصی از امتیاز داشته باشد و در رابط کاربری نمایش داده شود. هزینه‌های داخل پرانتزها برای نمونه و به عنوان پیشنهاد آورده شده‌اند.

5. سیستم امتیازدهی

امتیازدهی بر اساس زنجیره حذف‌ها به صورت تصاعدی افزایش می‌یابد:

- حذف اول: $10 \times$ تعداد مهره‌های حذف‌شده
- حذف دوم: $20 \times$ تعداد مهره‌های حذف‌شده
- حذف سوم: $40 \times$ تعداد مهره‌های حذف‌شده
- امتیاز حذف‌های بعدی به ترتیب هر بار دارای ضربی ۲ تا بیشتر از ضرب حذف قبلی است. این سیستم باعث تشویق بازیکن به ایجاد واکنش‌های زنجیره‌ای می‌شود.

6. نکات فنی و پیاده‌سازی

6.1. ساختار داده‌ها و مدیریت حافظه

- استفاده از `std::vector` برای پیاده‌سازی جدول و ابزارهایی مانند اشاره گر‌ها یا `struct` و `class` توصیه می‌شود.

6.2. ذخیره و بازیابی با فرمت `txt`

- بازی باید قابلیت ذخیره و بازیابی وضعیت را داشته باشد.
- اطلاعات شامل آرایش جدول، امتیاز فعلی، تجهیزات خریداری‌شده و زمان باقی‌مانده (و هر اطلاعاتی که برای از سرگیری مجدد بازی لازم است) باید در فایل `txt` ذخیره شوند.
- شروع برنامه باید منوی با گزینه‌های `New Game` و `Load Last Game` داشته باشد.

6.3. رابط کاربری (UI) تحت کنسول

- پس از هر حرکت، ترمینال باید پاک‌سازی و جدول جدید نمایش داده شود.
- برای تمایز مهره‌ها از کاراکترهای مختلف استفاده شود.
- امتیاز نهایی بازیکن، تجهیزات (قابل خرید) و کلیدهای کنترلی (مانند `save`) باید در کنار جدول نمایش داده شوند.
- حرکت انجام شده توسط بازیکن و به ازای هر واکنش زنجیره ایی امتیاز کسب شده به خاطر هر حذف (به ترتیب شامل حذف اول، دوم، ...) باید در کنار جدول نمایش داده شوند
- به ازای هر بار اعمال شدن جاذبه جدول جدید باید به مدت حداقل ۰/۵ ثانیه نمایش داده شود
- هر بار که حرکت جدیدی انجام می‌شود یا جاذبه اعمال می‌شود کاراکترهایی که در این مرحله حذف می‌شوند به صورت کاراکتر فاصله " " (یا به اصطلاح جای کاراکترهای حذف شده باید خالی شود) به مدت زمان حداقل ۰/۵ ثانیه قبل از اعمال جاذبه بعدی در جدول نمایش داده خواهند شد

```
=====
| GEM CASCADE: RECURSIVE EDITION | TIME: 02:14 | HIGH SCORE: 50,000 |
=====
| CURRENT SCORE: 12,450 | COMBO: x4 (INSANE!) |
=====

  0  1  2  3  4  5  6  7
  +--+--+--+--+--+--+--+
0 | $ | @ | # | % | $ | @ | ! | # |
  +--+--+--+--+--+--+--+
1 | @ | @ | @ | % | # | % | $ | ! | <-- (MATCH CLEARED!)
  +--+--+--+--+--+--+--+
2 | # | $ | % | ! | ! | ! | @ | % |
  +--+--+--+--+--+--+--+
3 | % | # | % | @ | $ | # | % | $ |
  +--+--+--+--+--+--+--+
4 | $ | ! | $ | % | @ | % | # | @ |
  +--+--+--+--+--+--+--+
5 | % | % | # | % | ! | $ | @ | ! |
  +--+--+--+--+--+--+--+

[GAME LOG & RECURSION TRACE]
> User swapped (1,2) with (1,3).
> Match Found: Row 1 (Type '@'). Removing 3 gems...
> Gravity Applied. Gems fell.
> RECURSION CHECK 1: Match Found! Col 5 (Type '!'). Removing 3 gems...
> CHAIN REACTION! Score Multiplier increased to x2.
> RECURSION CHECK 2: No matches. Board Stable.

[CONTROLS]: W/A/S/D: Move Cursor | Enter: Select | S: Save Game | Q: Quit
[INPUT] Enter Source Row & Col: _
```

نمایی از یک نمونه محیط کنسولی بازی شامل چند بخش از مواردگفته شده در فایل (و چند بخش اضافه!)

برای شهود بهتر شما

در انتخاب شکل جدول نمایش داده شده و نوع نمایش اطلاعات خواسته شده و... مختارید. برای دریافت نمره کامل صفحه بازی و نحوه استفاده از بازی باید کاملاً قابل فهم و دارای توضیحات لازم باشد.(مثل شکل)

7. نکات تکمیلی

- رعایت اصول کدنویسی تمیز (Clean Code) و مستندسازی الزامی است.
- استفاده از رنگ‌های ANSI برای تمایز کارکترها امتیاز مثبت دارد.
- طراحی ماژولار و تفکیک منطق بازی از رابط کاربری در ارزیابی نهایی مؤثر است.
- نمایش زمان باقی‌مانده بازی به صورت لحظه‌ای امتیاز مثبت دارد.